NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Software Engineering

BSCS Section 6E

Software Architecture Document

Plasma: The Social Intelligence Layer for Steam

Submitted by:

Tayyab Khalid - 23L-0501

Muhammad Wahaj - 23L-0560

Muhammad Saad - 23L-0679

Muhammad Ibrahim Alam - 23L-3075

Submitted to:

Mr. Durraiz Waseem

Date of submission: March 30, 2026

Table of Contents

1. Introduction

1.1 Purpose and Scope

This Software Architecture Document (SAD) provides a comprehensive architectural overview of **Plasma**, a web-based "Second Screen" social intelligence layer designed for the PC gaming ecosystem. Plasma bridges the gap between decentralized game launchers (Steam, Epic, Riot) and the social coordination required by modern gamers. The system acts as a centralized data aggregator, utilizing the Steam Web API for automated tracking and the IGDB API for manual non-Steam game tracking ("The Omni-Library").

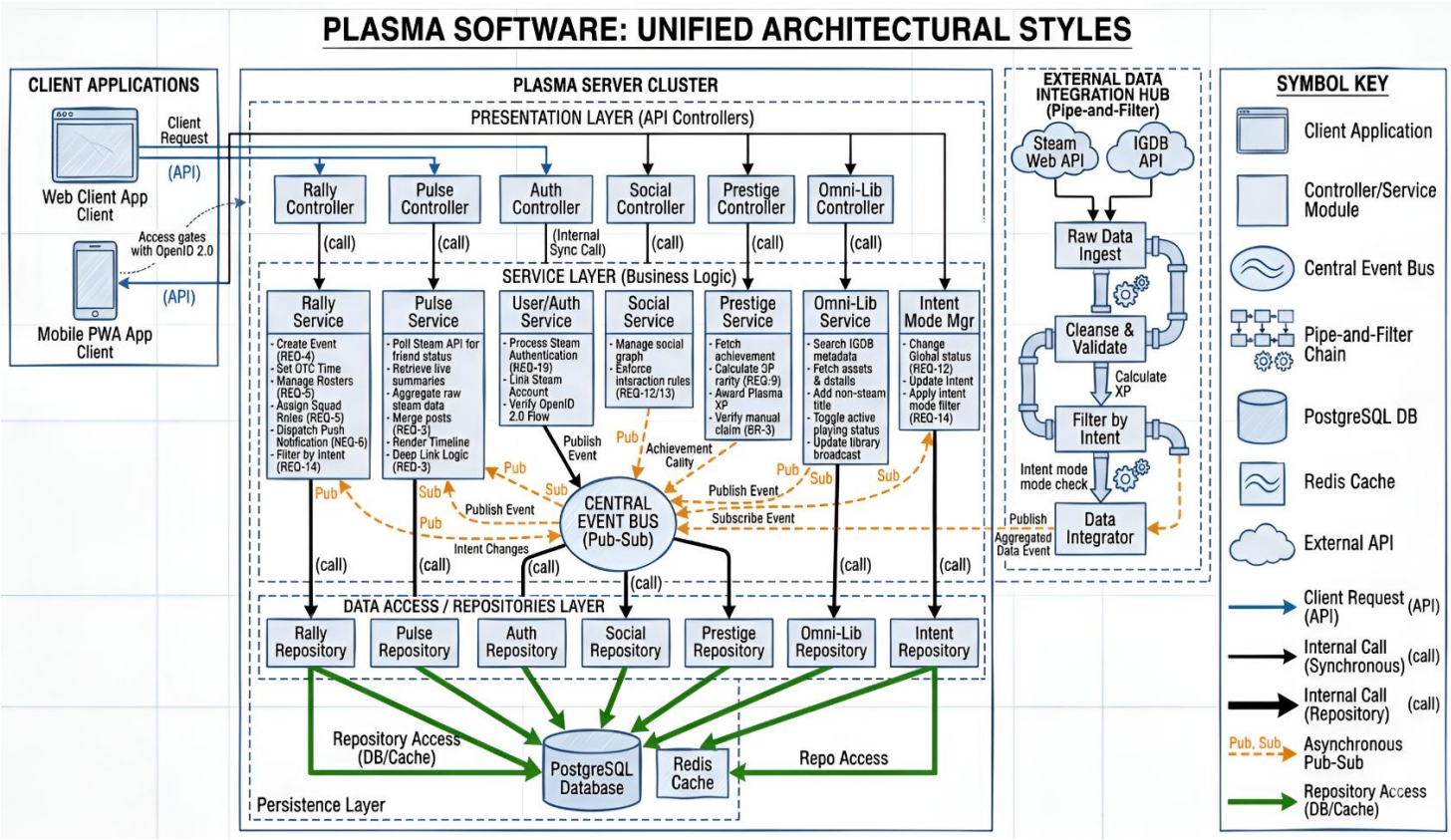
1.2 Architectural Goals

The architecture is designed to fulfill several rigorous constraints outlined in the system requirements:

- **Strict Rate Limiting:** Mitigating the strict API rate limits imposed by Valve Corporation through aggressive data caching.
- **Asynchronous Coordination:** Supporting real-time feed updates ("The Pulse") and delayed push notifications for event scheduling ("The Rally").
- **Contextual HCI:** Adapting the data presentation based on global "Intent Modes" (Competitive, Chill, Offline) to reduce cognitive load and social friction.
- **Secure Identity:** Enforcing a "Double Opt-In" security protocol for messaging and requiring OpenID 2.0 for verified Steam identity linkage.

2. Architectural Style Selection & Justification

To satisfy the highly diverse operational requirements of Plasma, the system utilizes a **Heterogeneous Architecture**. It unifies multiple distinct architectural styles, mapping specific patterns to the domains where they are most effective.



2.1 Client-Server Architecture

- **Application:** The macro-level structure of Plasma separates the front-end user interfaces ("Client Applications") from the centralized backend ("Plasma Server Cluster"). They communicate strictly via REST/WebSocket APIs over the network.
- **Justification:** This separation ensures the web SPA and mobile PWA remain lightweight and platform-agnostic. It centralizes heavy computational logic (like calculating Plasma XP) and sensitive security operations (OpenID 2.0 handshakes) securely on the server, ensuring clients cannot easily bypass the system's "Double Opt-In" rules.

2.2 Layered Architecture with the Repository Pattern

- **Application:** Inside the Server Cluster, the architecture is strictly horizontal: Presentation Layer -> Service Layer -> Data Access/Repositories Layer. The Persistence tier relies heavily on the **Repository Pattern** to interact with PostgreSQL and Redis.
- **Justification:** The Layered style enforces the Separation of Concerns (SoC). By specifically using the Repository Pattern at the base layer, the Business Logic (Service Layer) is completely decoupled from the actual SQL queries. If the database schema changes, or if caching strategies are updated, only the Repositories need to be modified, protecting the core social logic from breaking.

2.3 Publish-Subscribe (Event-Driven) Architecture

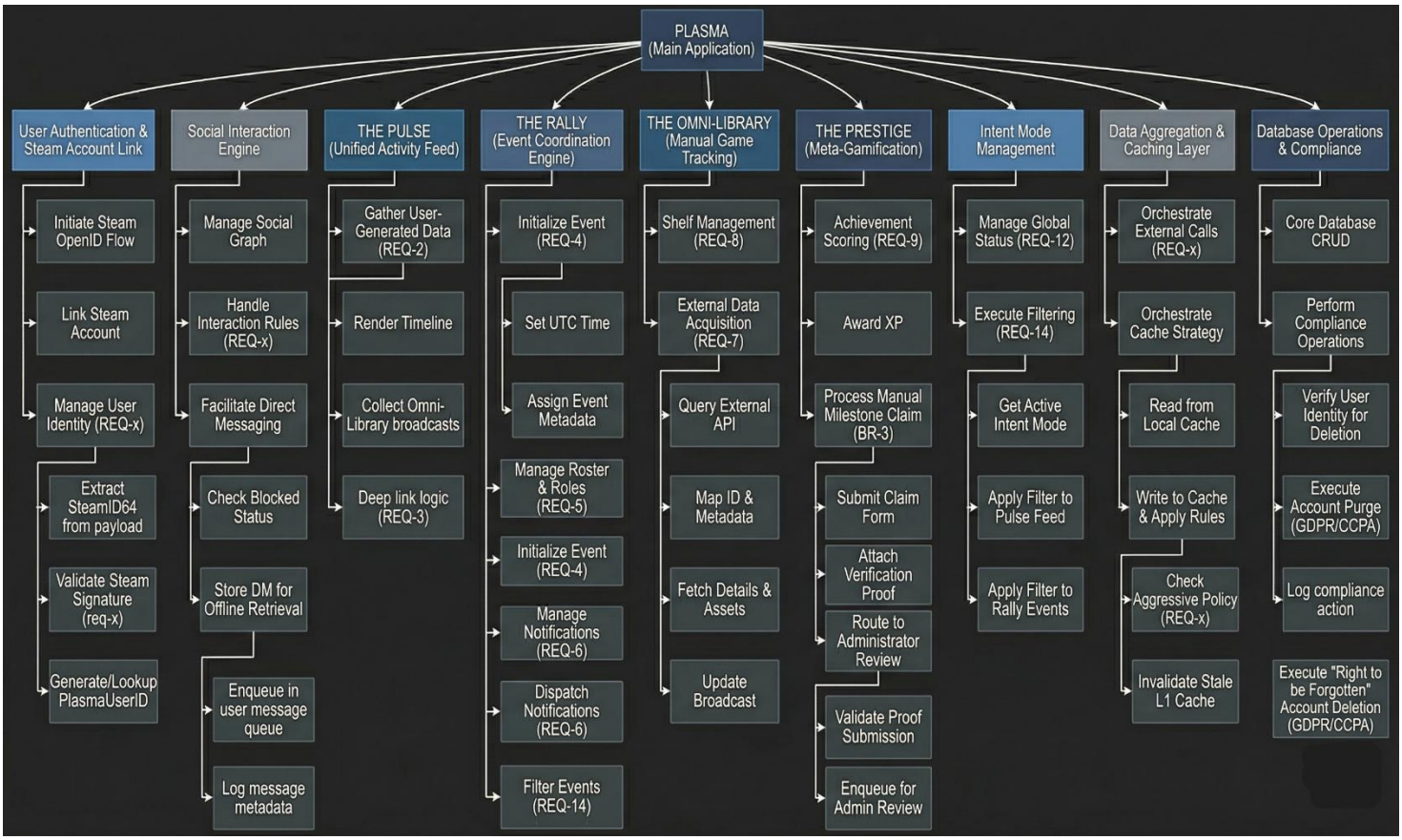
- **Application:** Internal communication between the decoupled service modules (Auth, Rally, Pulse, Prestige) operates on a Central Event Bus using an asynchronous Pub-Sub model.
- **Justification:** Plasma relies heavily on asynchronous social awareness. If a user RSVPs to a Rally event, synchronous calls would bottleneck the system. Instead, the Rally Service publishes a state-change event. Subscribed services (like the Notification Manager) listen for these events and react independently, such as dispatching a push notification without blocking the main execution thread.

2.4 Pipe-and-Filter Architecture

- **Application:** The External Data Integration Hub, responsible for polling the Steam Web API and querying the IGDB API, utilizes a Pipe-and-Filter chain.
- **Justification:** External gaming data arrives in raw, highly nested JSON formats. Data passes through sequential, independent filters: Raw Data Ingest -> Cleanse & Validate -> Filter by Intent -> Data Integrator. This ensures that only standardized, relevant data enters the Plasma ecosystem, drastically reducing database bloat and isolating API parsing logic from the core application.

3. Software Architecture Diagram

The functional decomposition model illustrates the hierarchical breakdown of the Plasma system into highly cohesive, loosely coupled modules. This modularity ensures that the system is resilient; an outage in the IGDB API will gracefully degrade the Omni-Library without crashing the Steam-powered Pulse feed.

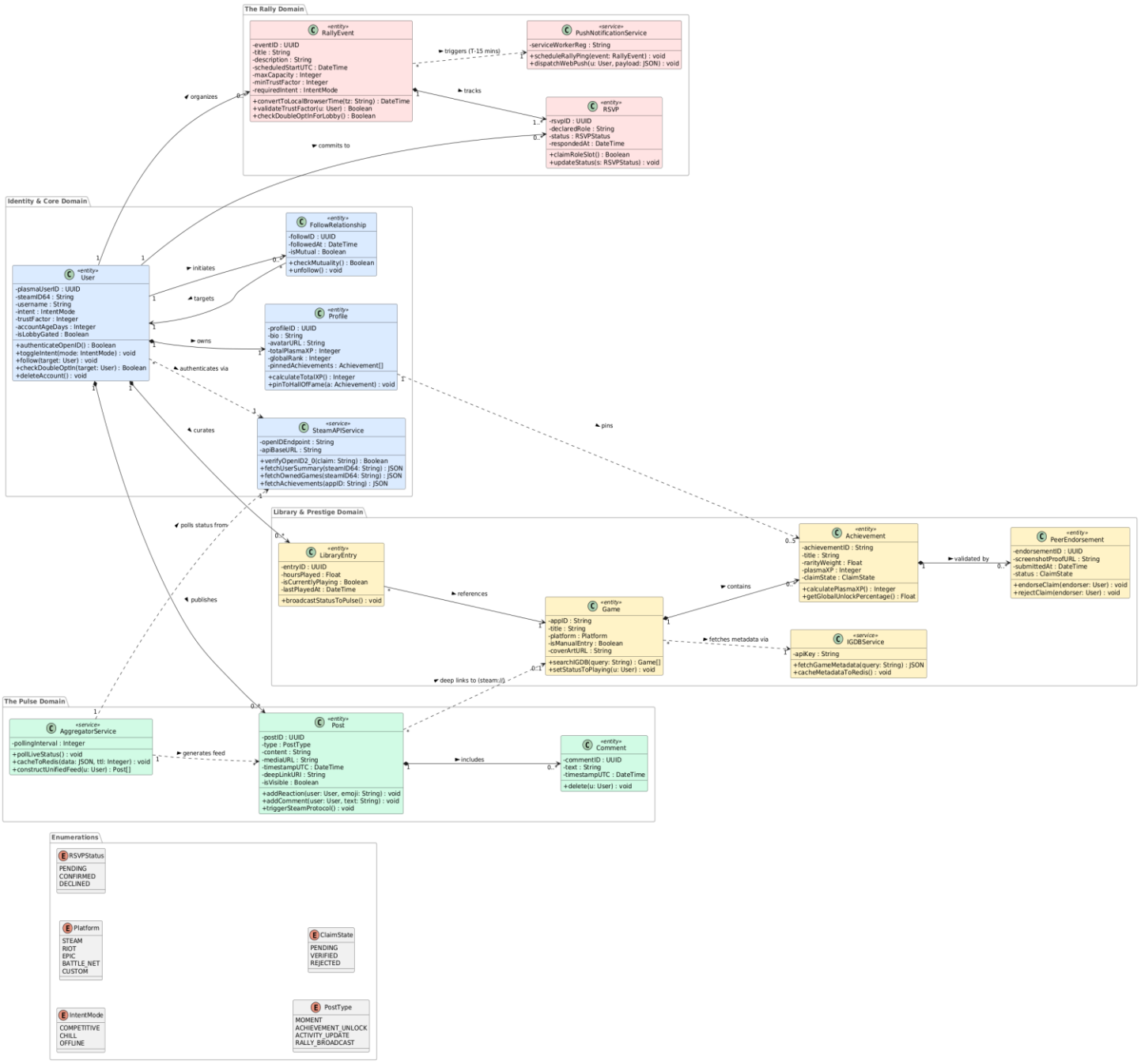


3.1 Subsystem Responsibilities

- **User Authentication & Steam Account Link:** Dedicated exclusively to managing the Valve OpenID 2.0 flow. It verifies the cryptographic signature of the Steam login, extracts the SteamID64, and maps it to the internal plasmaUserID.
- **The Pulse (Unified Activity Feed):** Operates as the social aggregator. It is responsible for rendering the timeline, fetching cached Steam status updates, merging them with user-generated "Moments," and executing the OS-level deep link logic.
- **The Rally (Event Coordination Engine):** Encapsulates all temporal and scheduling logic. It handles the critical task of converting stored UTC timestamps into the local browser timezones of the viewing users, manages role allocations (e.g., Tank, Healer, DPS), and evaluates the "Trust Factor" of RSVPs against the host's lobby gating rules.
- **The Omni-Library & The Prestige:** These modules manage static game data and player progression. The Prestige calculates the proprietary "Plasma XP" by applying a mathematical weight against the global rarity percentage of unlocked Steam achievements. It also handles the "Pending" state pipeline for peer-endorsed manual claims.
- **Database Operations & Compliance:** Centralizes CRUD operations and strictly enforces data privacy policies, notably executing the GDPR/CCPA "Right to be Forgotten" mandates by purging user data within 30 days of an account deletion request.

4. Refined Class Diagram

The refined domain model establishes the static structural blueprint of the Plasma system. It defines the exact data entities, their internal attributes, behavioral methods, and the strict mathematical multiplicities governing their relationships.



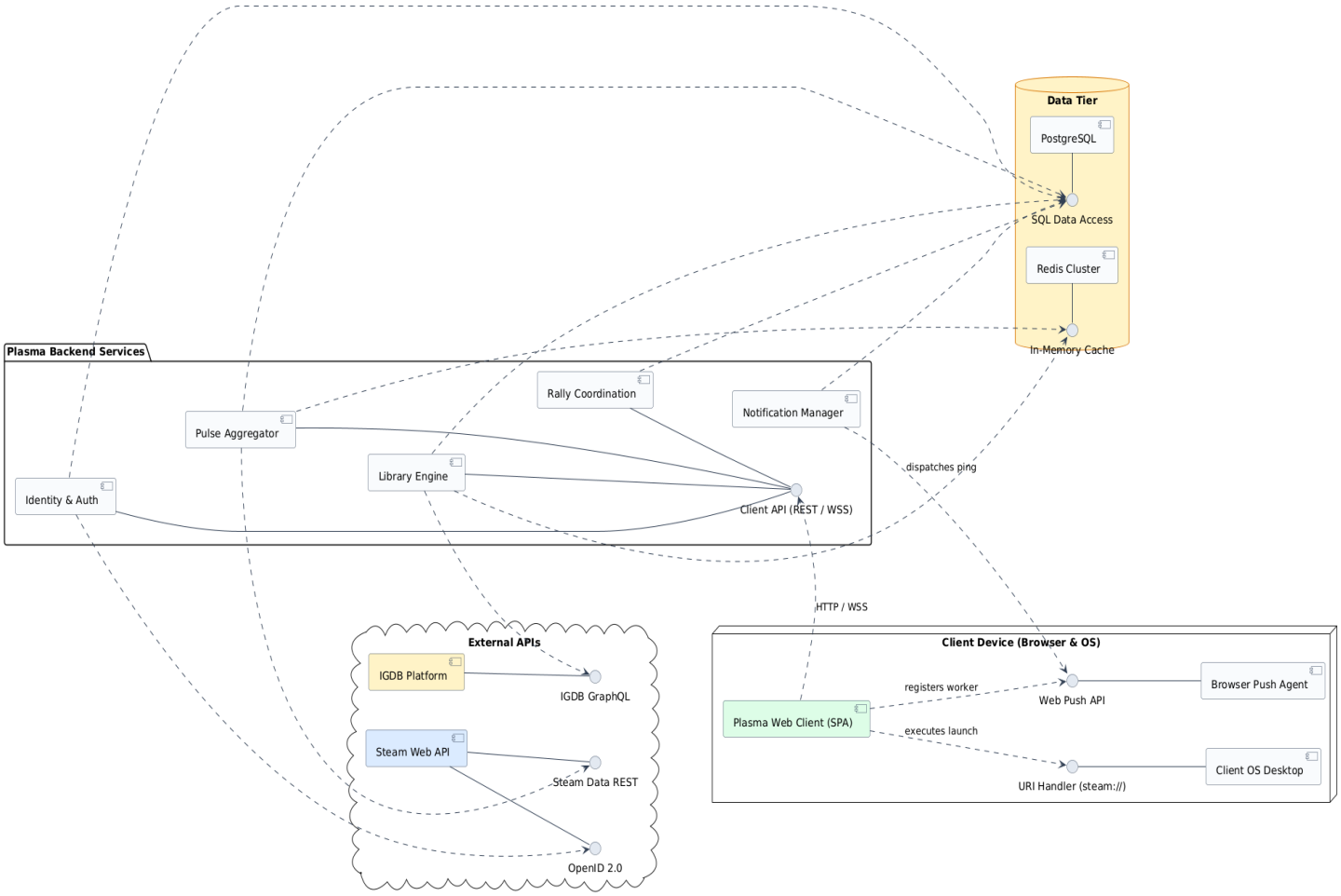
4.1 Structural Design & Multiplicity

- **Core Identity (Composition):** The User entity is the root aggregate. A User has a strict 1-to-1 composition relationship with a Profile. If a User is deleted, their Profile is cascaded and destroyed.
- **Social Graph (Directed Association):** The FollowRelationship acts as a bridging entity between two User instances to determine mutuality. The method checkMutuality() is critical, as it serves as the boolean gatekeeper for the system's Double Opt-In messaging requirements.

- **Event Tracking (Aggregation & Composition):** A User organizes (0..*) RallyEvent instances. However, the RallyEvent has a strong compositional relationship with RSVP (1..*), meaning an RSVP cannot exist without a parent event.
- **Gamification Constraints:** The diagram enforces the business rule that a Profile may pin exactly (0..5) Achievement objects to their "Hall of Fame." Furthermore, manual Achievement claims utilize the ClaimState enumeration (PENDING, VERIFIED, REJECTED), requiring validation via the PeerEndorsement entity.

5. Component Diagram (UML 2.0)

The component diagram illustrates the physical software microservices, the external API boundaries, and the precise interfaces required to wire the distributed system together. It utilizes strict UML 2.0 ball-and-socket notation.



5.1 Interface Specifications and Integrations

- **The Client API Boundary:** The entire suite of Plasma Backend Services hides its internal complexity behind a unified Client API (REST / WSS) provided interface. The Plasma Web Client (SPA) consumes this interface via standard HTTPS and WebSockets (for live Pulse feed polling).
- **Data Tier Encapsulation:** To optimize performance and adhere to external rate limits, the data tier is heavily abstracted. Backend services require the SQL Data Access interface for persistent, relational storage (PostgreSQL) regarding user graphs and schedules. Concurrently, the Pulse Aggregator and Library Engine heavily depend on the In-Memory Cache interface (Redis Cluster) to temporarily store transient IGDB metadata and Steam live-status checks, shielding the application from rate-limit blacklisting.

- **External API Sockets:** The backend explicitly defines required dependencies on external platforms. The Identity & Auth component requires the OpenID 2.0 interface, while the Pulse Aggregator taps into the Steam Data REST API.
- **Host OS Handoff:** Fulfilling the localized "Second Screen" architectural requirement, the diagram uniquely models the client device's Operating System as a component. The Plasma Web Client requires a URI Handler interface to pass Steam execution commands directly to the user's desktop environment, smoothly transitioning the user from the web platform into the game client.